Executive read & Review

The concept is good. More than that, it is pointed at a real product gap.

I would not reduce this to “Figma Lite.” The better product is:

> **A package-backed GUI workbench for application interfaces, where visual editing, validation, documentation, app adapters, and AI/developer handoff all orbit one structured representation of interface intent.**

The key product object is not the canvas.
The key product object is the **GUI Package**.

The canvas/editor is how humans shape the package.
The validator is how the package gets tested.
The docs/AI briefs are how the package travels.
The GeoSee adapter is how the package becomes useful inside the actual app.

That matches the docs: the current thesis is “implementation-ready interface intent,” not simply drawing screens.

My product recommendation

I would productize this as two tightly related things:

1. GeoSee GUI Lab
   A first-party internal tool for shaping, testing, and refining GeoSee panels.

2. GUI Workbench / Panel Foundry
   A portable standalone product for application GUI authoring, validation, packaging, and handoff.

The GeoSee GUI Lab proves the concept in a real, difficult host app. The standalone product is the generalizable form.

The important distinction:

GeoSee GUI Lab = first proving ground and internal power tool.
GUI Workbench = reusable product architecture.
Panel Foundry = possible sharper name for the first public-facing product.

I would keep **GUI Workbench** as the internal umbrella name for now, because the glossary already uses it as the broad term for standalone and embedded modes. I would use **Panel Foundry** as the more product-like name for the focused editor experience, especially if the first product surface is panels, tool containers, inspectors, sidebars, and presentation controls.

What the concept really is

The product is not a replacement for Figma, Storybook, Retool, Builder, or a code agent.

It sits between them.

Figma already has plugin APIs that can read/write design files, and Code Connect bridges design components to code components for developer and AI-agent context. (Figma Developer Docs) Storybook already occupies the "build, test, and document components in isolation" space. (Storybook) Figma Make is now explicitly positioned around AI-assisted functional prototypes and app-like outputs. (Figma)

So I would not position GUI Workbench as a generic design tool. That is a losing framing.

I would position it as:

> **A structured interface-intent workbench for real app panels, with deterministic validation, context profiles, host-app adapters, package export, and safe embedded refinement.**

That is much more specific and more defensible.

Strongest product insight

The strongest idea in the whole package is this chain:

rough interface need
→ guided panel structure
→ visual refinement
→ device/use-case/input/background validation
→ GUI Package
→ implementation brief / AI-agent brief / host-app adapter
→ optional safe embedded apply

That chain solves a real GeoSee problem: you are building a dense technical visualization product where UI changes are hard to communicate, hard to test over real scene backgrounds, and risky when handed to AI/coding agents vaguely.

The GOC/GeoSee context file is especially strong because it identifies exactly why this matters: GeoSee has complex visual backgrounds, technical and non-technical modes, project-specific variants, presentation needs, and lots of real panel types.

What I would do as a product

I would make the product schema-first, not canvas-first.

The mistake would be building a pretty editor where the visual canvas becomes the source of truth. That leads to Figma-clone gravity.

The better model:

GUI Package / schema is source of truth.
Editor modifies the package.
Preview renders the package.
Validation reads the package.
Docs are generated from the package.
GeoSee adapter maps allowed package fields to app surfaces.

That gives you a product that is portable, inspectable, versionable, and usable by AI agents. The package concept note already points in this direction: the GUI Package carries interface intent, validation context, project overrides, documentation, AI-agent briefs, and app-adapter compatibility.

I would make the first product promise:

> “Create and refine real app control panels with validation and implementation handoff.”

Then the second promise:

> “Safely tune approved GeoSee GUI surfaces inside the live app.”

Then the third promise:

> “Export/import GUI packages across apps, projects, agents, and documentation workflows.”

What you may be missing

1. A clear binding-intent model

The docs correctly say the workbench should not own renderer logic, data pipelines, or app state. But the product still needs a way for controls to express what they intend to bind to.

Example:

```
{
  "controlId": "terrain-opacity-slider",
  "type": "slider",
  "label": "Terrain opacity",
  "bindingIntent": {
    "domain": "layer",
    "target": "terrain",
    "property": "opacity",
    "valueType": "number",
    "range": [0, 1],
    "hostBindingRequired": true
  }
}
```

This does **not** implement the logic. It tells the developer, AI-agent, or GeoSee adapter what the control is supposed to affect.

Without this, the package risks becoming only visual. With it, the package becomes implementation-ready without becoming dangerous.

2. A formal surface registry

The host-app adapter should expose a registry of editable GUI surfaces.

For GeoSee, that might look like:

```
surface: layer-control-panel
status: editable
allowed edits:
  - labels
  - section order
  - default collapsed state
  - optional control visibility
```

```
  - width within range
  - spacing tokens
locked:
  - data bindings
  - renderer calls
  - project save logic
save targets:
  - draft
  - project override
  - exported package
```

The safe editing charter already defines safe and restricted classes of edits, but the actual product needs that concept formalized per surface.

3. A compatibility and migration model

The package docs mention compatibility, but this should become a core product feature, not a later afterthought.

Every package should declare:

```
{
  "schemaVersion": "0.1.0",
  "workbenchVersion": "0.1.0",
  "targetHost": "geosee",
  "targetHostVersion": "0.8.x",
  "adapter": "geosee-gui-adapter",
  "adapterVersion": "0.1.0",
  "requiredComponents": ["button", "slider", "toggle", "collapsible-section"],
  "unsupportedFeatures": []
}
```

This matters because standalone packages, embedded packages, project overrides, and AI briefs will drift over time.

4. A state model for components

The docs mention states, but the product should make component states a first-class part of editing and validation.

Every meaningful control should support:

default
hover
focus
pressed
active
disabled
loading
warning
error
selected
empty
invalid

GeoSee will need this because technical tools often have unavailable data, loading states, disabled controls, warning states, and missing-source states.

5. A panel behavior model, not just panel layout

The editor should define things like:

dock left / dock right / floating
resizable / fixed
min width / max width
scroll behavior
sticky header
collapsible sections
responsive stack rules
z-index / overlay rules
safe edge clearance
presentation-mode simplification

A Figma-like editor that only positions rectangles will not be enough. GeoSee needs app-panel behavior.

6. A validation baseline tied to real standards

The docs correctly leave numeric validation values unresolved. You should eventually define profile-driven defaults using known standards and platform conventions. WCAG 2.2's target-size minimum is about reducing accidental activation of nearby targets and defines a 24 by 24 CSS pixel minimum with exceptions, while WCAG target-size enhanced uses 44 by 44 CSS pixels. (W3C) Material Design recommends touch targets of at least 48 by 48 dp. (Material Design)

Apple's button guidance says a button generally needs at least a 44 by 44 pt hit region. (Apple Developer) WCAG contrast guidance is also directly relevant for text/background readability. (W3C)

For GeoSee, I would use those as configurable baselines, not hard universal laws.

7. A manual-plus-automated background readability model

The docs correctly identify background stress testing as essential, especially over terrain, satellite imagery, geology overlays, noisy images, moving scenes, and transparent panels.

But automated contrast checks alone will fail over complex scenes. I would make this hybrid:

automated:
- sample contrast under text/control regions
- detect extreme local contrast/noise
- detect transparency risk
- flag low-contrast overlays

manual:
- reviewer marks pass/fail per background
- reviewer can acknowledge risk
- screenshots stored in package
- unresolved warnings travel into AI/dev brief

That keeps it honest.

8. Governance for project overrides

The docs already say project overrides should not silently mutate the base GeoSee GUI. The missing product layer is promotion governance:

project override
→ repeated in 2+ projects
→ candidate base pattern
→ reviewed
→ promoted to GeoSee default
→ old overrides migrated or left pinned

Without this, overrides become messy forks.

9. A “reference asset” policy

The package docs say assets should support interface testing, not become hidden app dependencies. I would make that explicit in the package format.

Assets are allowed as:

preview backgrounds
reference screenshots
stress-test videos
thumbnails
comparison captures

Assets are not allowed as:

secret project data
runtime dependencies
API keys
hidden app state
confidential datasets unless deliberately attached

Opportunities

1. GeoSee can become the flagship proof

GeoSee is a very strong proving ground because it has dense technical panels, complex visual backgrounds, presentation modes, project-specific needs, and AI-agent handoff risk. The docs already identify that if the workbench handles GeoSee, it should generalize to other technical visualization products.

That is a good product wedge.

2. Technical visualization GUI is underserved

Figma and Penpot are broad design platforms. Storybook is developer/component-centered. tldraw is a strong canvas SDK if you need custom canvas infrastructure, but it is not itself a domain-aware panel validation product. (Penpot)

GeoSee’s need is narrower and sharper:

technical app panels
over complex scenes
with boardroom/presentation variants
with safe in-app refinement
with implementation handoff

That is a real lane.

3. AI-agent handoff is a major product angle

The docs are right to treat AI-agent briefs as first-class outputs. They should include target surface, component hierarchy, required controls, layout rules, state assumptions, validation findings, acceptance criteria, no-touch areas, and implementation notes.

This is valuable because it reduces vague “make the UI better” prompts.

4. Validation can be a differentiator

Most lightweight UI editors do not deeply validate against:

device profile
display profile
input method
use case
presentation mode
complex background
host-app safety
project override status

Your docs already have this philosophy. Make that central.

5. Standalone-to-embedded transfer is powerful

A package that can move from standalone design into GeoSee, and from GeoSee back out to standalone review, is a strong product loop. The package note already defines both standalone and embedded workflows.

That loop is more interesting than either mode alone.

Challenges

1. Figma-clone gravity

The second you say “Figma Lite,” people will expect freeform drawing, layers, multiplayer, comments, variants, auto layout, prototyping, design libraries, and dev handoff.

You do not want that whole surface.

The editor should feel Figma-like only in the parts that matter:

canvas
selection
drag/reorder
properties inspector
component palette
layers/tree
tokens
preview
export

It should not become a general design suite.

2. Schema/editor/render drift

If the schema says one thing, the editor shows another, and GeoSee renders a third thing, the product dies.

The architecture must keep one source of truth:

GUI Package schema
→ editor state
→ preview renderer
→ validation
→ export
→ host adapter

3. Embedded safety is real

Embedded mode is powerful and risky. The safe editing charter is correct: the workbench should edit only host-exposed interface presentation surfaces and must not edit core logic, rendering, permissions, destructive behavior, authentication, project file handling, external integrations, or host app state architecture.

This cannot be hand-wavy. GeoSee must define the boundary.

4. Complex background validation is hard

Transparent panels over satellite imagery, point clouds, geology overlays, terrain, and moving 3D scenes are not solved by simple text contrast rules. This will need stress-test previews, screenshot capture, partial automated heuristics, and human approval.

5. Non-developer users still need guardrails

A domain expert should be able to shape a seismic tool panel, but not accidentally create a nonsense data-binding model, destroy hierarchy, or expose controls that should not exist in presentation mode.

The wizard and archetypes are important here.

6. Overfitting to GeoSee

GeoSee-specific panel types should live in the GeoSee adapter, GeoSee package examples, GeoSee archetypes, GeoSee validation presets, and GeoSee background assets. The Workbench Core should remain app-agnostic. The docs already call this out directly.

Advised full architecture

I would structure the architecture like this:

GUI Workbench Platform
│
├─ 1. Workbench Core
│ ├─ schema system
│ ├─ design token system
│ ├─ component model
│ ├─ panel/container model
│ ├─ use-case/device/input/profile model
│ ├─ validation engine
│ ├─ preview state model
│ ├─ package reader/writer
│ ├─ documentation generator
│ └─ optional AI review interface
│
├─ 2. Visual Editor Layer

```
│   ├─ canvas
│   ├─ component palette
│   ├─ panel tree / layer tree
│   ├─ properties inspector
│   ├─ token editor
│   ├─ profile selector
│   ├─ validation panel
│   └─ export panel
│
├─ 3. Preview / Stress-Test Layer
│   ├─ isolated preview
│   ├─ device/display preview
│   ├─ input-mode preview
│   ├─ boardroom/projector preview
│   ├─ background image/video preview
│   └─ host-app contextual preview
│
├─ 4. GUI Package System
│   ├─ manifest
│   ├─ tokens
│   ├─ components
│   ├─ panels
│   ├─ profiles
│   ├─ validation findings
│   ├─ project overrides
│   ├─ reference assets
│   ├─ generated docs
│   └─ change history
│
├─ 5. Host-App Adapter Layer
│   ├─ surface registry
│   ├─ component mapping
│   ├─ token mapping
│   ├─ allowed edit rules
│   ├─ locked surface rules
│   ├─ save/apply rules
│   ├─ permissions
│   ├─ compatibility declaration
│   └─ change log destination
│
├─ 6. Shells
│   ├─ Standalone Shell
│   └─ Embedded Shell
│
```

└─ 7. Optional Integrations
   ├─ Figma export/import
   ├─ Storybook export
   ├─ code component mapping
   ├─ AI-agent brief export
   ├─ screenshot/reference import
   └─ future open-source adapter SDK

This is consistent with the document architecture: Workbench Core, Standalone Shell, Embedded Shell, Host-App Adapter, GUI Package, Validation Engine, Preview Engine, Documentation/Export Engine, and optional AI review.

GeoSee-specific architecture

For GeoSee, I would build:

GeoSee
│
├─ GeoSee GUI Adapter
│ ├─ editable surface registry
│ ├─ locked surface registry
│ ├─ GeoSee component map
│ ├─ GeoSee token map
│ ├─ project override manager
│ ├─ save/apply policy
│ ├─ validation profile presets
│ └─ scene background capture hooks
│
├─ Embedded GUI Lab Route
│ ├─ select editable panel
│ ├─ edit safe properties
│ ├─ preview in app context
│ ├─ run validation
│ ├─ save draft
│ ├─ save project override
│ └─ export package/brief
│
├─ Project GUI Overrides
│ ├─ woodjam.gui.json
│ ├─ bingham.gui.json
│ ├─ utah-forge.gui.json
│ └─ kliyul.gui.json

```
│
└─ Base GeoSee GUI System
   ├─ base tokens
   ├─ base components
   ├─ base panels
   └─ base validation profiles
```

The first GeoSee surfaces I would expose are:

1. Layer Control Panel
2. Lighting Control Panel
3. Presentation Control Panel
4. Camera / Navigation Panel
5. Map Overlay Controller

I would delay:

seismic/geophysical tool container
drillhole inspector
data import container
assay interval editor

Those are more domain/data-bound. They are important, but they risk pulling the workbench into application logic too early.

GeoSee safe edits should include labels, section order, spacing, panel width, default collapsed state, component variants, optional control visibility, project presentation settings, layer panel defaults, and tool panel layout. GeoSee locked areas should include renderer internals, geological data processing, geospatial parsing, save/load logic, authentication, permissions, external data integrations, destructive actions, app state architecture, camera/rendering engine behavior, and mining/geoscience calculation logic.

Standalone product architecture

The standalone product should not require GeoSee to be running.

It should have:

```
Standalone GUI Workbench
│
├─ Project workspace
```

```
├─ Wizard
├─ Panel editor
├─ Component library
├─ Token editor
├─ Archetype library
├─ Device/display/input profile selector
├─ Background stress-test library
├─ Validation runner
├─ Package export/import
├─ Documentation export
├─ AI-agent brief export
└─ Optional host kits
   ├─ Generic Web App Kit
   ├─ GeoSee Kit
   ├─ Scientific Visualization Kit
   └─ Dashboard/Internal Tools Kit
```

The host kits matter because they keep the core portable while still letting different domains feel concrete.

Recommended package format

I would use a **folder-based package** first, with JSON as the primary structured format and Markdown for human-readable generated docs.

Example:

```
geosee_layer_controls_gui_package_v0_1/
  manifest.json
  tokens.json
  components.json
  panels.json
  profiles.json
  validation.rules.json
  validation.report.json
  overrides/
    woodjam.override.json
    utah-forge.override.json
  docs/
    implementation_brief.md
    ai_agent_brief.md
    validation_report.md
```

```
  package_summary.md
assets/
  backgrounds/
  screenshots/
  preview_exports/
```

Why folder-based first:

- readable in Git
- easy to diff
- easy to inspect
- easy to attach assets
- easy for AI agents to consume
- easy to zip later
- avoids opaque binary format too early

Later, it can be exported as:

.gui-package.zip

But the internal structure should remain readable.

Advised first build

Build 1 — GeoSee GUI Lab / Panel Foundry Alpha

This should be a **standalone package-backed editor** focused on quickly refining real GeoSee GUI panels.

Not a static document generator.
Not full embedded mode.
Not full Figma.
But yes: a small Figma-like editor for real app panels.

Goal

Create a working visual editor where you can build/refine one real GeoSee panel, validate it against selected profiles, preview it over GeoSee-like backgrounds, and export a package plus implementation/AI-agent brief.

First panel

I would choose **Layer Control Panel**.

Why:

- central to GeoSee
- generic enough to generalize
- real enough to matter
- includes visibility, opacity, grouping, ordering, warning states
- useful in technical mode and presentation mode
- does not require deep geoscience processing logic

Lighting Control Panel is easier, but slightly too shallow. Seismic Tool Container is more impressive, but too likely to drag in data and domain logic early.

Build 1 in-scope

1. GUI Package v0.1 folder format
2. Package manifest
3. Basic token file
4. Basic component model
5. Basic panel/container model
6. Layer Control Panel archetype
7. Component palette:
   - button
   - icon button
   - toggle
   - checkbox
   - slider
   - dropdown
   - badge
   - warning row
   - collapsible section
   - panel header
8. Figma-lite editor:
   - select component
   - drag/reorder sections
   - add/remove controls
   - edit labels
   - adjust panel width
   - adjust spacing
   - set collapsed defaults
   - change variants

9. Properties inspector
10. Panel tree/layers list
11. Preview profiles:
    - desktop technical
    - tablet review
    - boardroom/projector
12. Background stress previews:
    - dark clean
    - light clean
    - GeoSee terrain screenshot
    - dense geology/satellite/technical screenshot
    - stripe/noise test
13. Deterministic validation:
    - missing labels
    - touch target warning
    - spacing warning
    - panel overflow
    - text truncation
    - font-size warning
    - hover-on-touch warning
    - simple contrast check
    - background readability manual flag
14. Export:
    - GUI package
    - implementation brief
    - AI-agent brief
    - validation report
    - preview screenshots

Build 1 out-of-scope

- embedded saving into GeoSee
- editing renderer or scene behavior
- live data binding
- code mutation
- full app workflow editing
- multiplayer
- plugin marketplace
- full Figma drawing tools
- AI making changes automatically
- screenshot-to-UI magic import

Build 1 success test

A non-front-end specialist should be able to open the tool, choose the Layer Control Panel archetype, adjust labels/sections/spacing/control visibility, preview it against desktop/tablet/boardroom profiles, see validation warnings, and export a package that a developer or AI coding agent can act on.

That directly matches the docs' stated success direction: create usable panels, preview them under realistic conditions, get validation warnings, refine without writing code, and export clear implementation/AI/developer handoff.

Advised second build bounds

Build 2 — GeoSee Adapter + Embedded Refinement Console

This is where your "Figma-lite editor for quickly refining the GeoSee GUI" becomes app-connected.

The second build should not expand into every integration. It should connect the editor to GeoSee safely.

Goal

Allow approved GeoSee GUI surfaces to be loaded, refined, validated, and saved as drafts or project overrides from inside GeoSee.

Build 2 in-scope

1. GeoSee Host-App Adapter v0.1
2. Surface registry for:
   - Layer Control Panel
   - Lighting Control Panel
   - Presentation Control Panel
3. Component mapping:
   - workbench button → GeoSee button
   - workbench slider → GeoSee slider
   - workbench toggle → GeoSee toggle
   - workbench section → GeoSee collapsible section
4. Token mapping:
   - font sizes
   - spacing
   - panel width
   - opacity

   - border radius
   - control heights
5. Embedded editor route:
   - open GeoSee GUI Lab
   - choose editable surface
   - inspect current surface
   - edit safe fields only
6. Save states:
   - draft
   - validated draft
   - project override
   - exported package
7. Apply rules:
   - apply to current project only
   - no base-system promotion yet unless manually reviewed
8. Validation inside GeoSee:
   - desktop/tablet/boardroom profiles
   - touch warnings
   - panel overflow
   - background stress with current scene capture
9. Change log:
   - who/when/what
   - previous value
   - new value
   - validation status
   - unresolved warnings
10. Round trip:
   - export from GeoSee to package
   - open in standalone
   - edit
   - reimport as draft/project override

Build 2 out-of-scope

- editing base GeoSee GUI without review
- editing data/rendering/camera engine internals
- changing project save/load logic
- changing tool calculations
- changing destructive action behavior
- live arbitrary code generation
- AI applying changes
- general Figma import
- Storybook integration
- public open-source release

Why this is the right second build

It gives you the fast GeoSee GUI refinement you want, while keeping the workbench safe. The embedded shell should respect adapter rules, load only host-exposed surfaces, edit permitted fields, run host-app validation, save drafts/project overrides, and produce change logs.

Integration and expansion roadmap

After Build 2, I would add integrations in this order:

1. Storybook exporter

Export Workbench panels/components as Storybook stories or docs pages. Storybook is already a known environment for building, testing, and documenting components in isolation. (Storybook)

This helps developers test component states outside GeoSee.

2. Figma export, not Figma dependency

Export package panels to Figma frames for visual review.

Do not make Figma the source of truth. The package remains the source of truth.

3. Figma import as reference only

Imported Figma frames should initially become reference assets or semi-structured sketches, not authoritative package objects. Figma's plugin ecosystem and Code Connect already handle strong design-tool workflows, so GUI Workbench should integrate where useful without becoming subordinate to Figma. (Figma Developer Docs)

4. AI review layer

Add AI critique only after deterministic validation is solid.

AI can summarize findings, critique density, suggest simplification, and draft implementation briefs. It should not override validation, rewrite package state silently, or apply embedded changes. That boundary is already clearly stated in the validation and safe-editing docs.

5. Canvas SDK evaluation

If you want to accelerate the Figma-lite editor, evaluate tldraw as a canvas foundation. It provides a React SDK with canvas infrastructure, selection/transformation, custom shapes, UI customization, and programmatic control. (Tldraw)

But I would only use it if the licensing and technical fit are acceptable. The risk is inheriting whiteboard assumptions you do not need.

6. Open-source core

Open-source later should probably start with:

- package schema
- validation rule model
- adapter contract
- standalone minimal shell
- example host app adapter

Do not open-source GeoSee-specific adapters, project data, proprietary component mappings, or business-sensitive panel archetypes until they are cleaned.

Product positioning

I would describe the product like this:

> **GUI Workbench is a package-backed interface workbench for application panels and tool containers. It lets teams create, refine, validate, document, and export real GUI structures for implementation or safe host-app integration.**

For GeoSee:

> **GeoSee GUI Lab is the first embedded use of GUI Workbench: a safe internal editor for refining approved GeoSee panels, presentation controls, layer controls, and project-specific GUI overrides without touching renderer, data, or application logic.**

For the Figma-lite editor:

> **Panel Foundry is the visual editing surface inside GUI Workbench: a lightweight, structured editor for arranging app panels, controls, sections, states, tokens, and responsive/profile variants.**

That gives you three layers of language:

GUI Workbench = system
Panel Foundry = visual editor/product surface
GeoSee GUI Lab = first host-app implementation

My recommended decision posture

Do not shrink the concept. It should remain broad as a knowledge capture.

But when you later choose build bounds, protect these principles:

1. Package/schema is the spine.
2. Visual editor is important, but not the source of truth.
3. GeoSee proves the system, but does not define the core.
4. Embedded mode must be adapter-governed.
5. Validation is a product feature, not a QA afterthought.
6. AI assists; it does not decide or apply.
7. Project overrides are explicit, reversible, and traceable.
8. Figma-like means fast visual refinement, not full design-suite cloning.

Bottom-line recommendation

As a product, I would build toward:

A structured, package-backed, Figma-lite workbench for real application GUI panels, with deterministic validation, background stress testing, project overrides, AI/developer handoff, and safe embedded host-app refinement.

First build:

Standalone GeoSee GUI Lab / Panel Foundry Alpha
→ package-backed visual editor
→ Layer Control Panel
→ validation

→ background stress preview
→ export package + implementation/AI brief

Second build:

GeoSee Adapter + Embedded Refinement Console
→ load approved GeoSee surfaces
→ edit safe properties
→ validate in app context
→ save drafts/project overrides
→ round-trip packages
→ no renderer/data/app-logic mutation

That gives you the Figma-lite GUI refinement path you want without collapsing the whole concept into a canvas tool or a risky live app mutator.